Function Point Analysis of Backend Functions for Bill Splitter Application

Lucy Gavrich - CS 320 - March 29, 2026

ORM Models

`UserManager`

```
create_user(self: UserManager, name: string,
            email: string, password: string)
```

Create a new User.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, name, email, password	4	3	4	6	12
EOs	New `User`	1	4	5	7	4
EQs	None	0	3	4	6	0
ILFs	`User` table	1	7	10	15	7
EIFs	None	0	5	7	10	0
Total						23

HouseholdManager

`create_household(self: HouseholdManager, creator: User, household_name: string)`

Create a new `Household`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, initial member, name	3	3	4	6	9
EOs	New `Household`	1	4	5	7	4
EQs	None	0	3	4	6	0
ILFs	`Household` table	1	7	10	15	7
EIFs	None	0	5	7	10	0
Total						20

Household

`add_user(self: Household, user: User)`

Add a `User` to an existing `Household`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, user	2	3	4	6	8
EOs	Updated `Household`	1	4	5	7	5
EQs	None	0	3	4	6	0
ILFs	`Household` table	1	7	10	15	10
EIFs	None	0	5	7	10	0
Total						23

`remove_user(self: Household, user: User)`

Remove a `User` from a `Household`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, user	2	3	4	6	8
EOs	Updated `Household`	1	4	5	7	5
EQs	None	0	3	4	6	0
ILFs	`Household` table	1	7	10	15	10
EIFs	None	0	5	7	10	0
Total						23

`BillManager`

`create_bill(self: BillManager, household: Household, biller: User, debts: dict[int, float], date: Date)`

Create a new `Bill` and associated `Debts`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, household, biller, dict of billees and amounts, date	5	3	4	6	20
EOs	New `Bill`	1	4	5	7	5
EQs	None	0	3	4	6	0
ILFs	`Bill` table	1	7	10	15	10
EIFs	`Debt` table	1	5	7	10	7
Total						42

Bill

update(self: Bill)

Update a Bill (called when a Debt is resolved). If all Debts are resolved, the Bill is marked as resolved.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self	1	3	4	6	4
EOs	Updated Bill	1	4	5	7	5
EQs	Outstanding Debts	1	3	4	6	4
ILFs	Bill table	1	7	10	15	10
EIFs	Debt table	1	5	7	10	7
Total						30

DebtManager

create_debt(self: DebtManager, bill: Bill, ower: User, amount: float)

Create a new Debt (called when a Bill is created).

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, bill, ower, amount	4	3	4	6	12
EOs	New Debt	1	4	5	7	4
EQs	None	0	3	4	6	0
ILFs	Debt table	1	7	10	15	7
EIFs	None	0	5	7	10	0
Total						23

Debt

`update(self: Debt, payment: Payment)`

Update a `Debt` (called when a `Payment` is made). Subtracts `Payment` amount from `Debt` amount. If `Debt` becomes zero, marks `Debt` as resolved and calls `Bill.update`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, payment	2	3	4	6	8
EOs	Updated `Debt`	1	4	5	7	5
EQs	Calls `Bill.update`	1	3	4	6	4
ILFs	`Debt` table	1	7	10	15	10
EIFs	`Bill` table	1	5	7	10	7
Total						34

PaymentManager

`create_payment(self: PaymentManager, debt: Debt, amount: float, date: Date)`

Create a new `Payment`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, debt, amount, date	4	3	4	6	16
EOs	New `Payment`	1	4	5	7	5
EQs	Calls `Debt.update`	1	3	4	6	4
ILFs	`Payment` table	1	7	10	15	10
EIFs	`Debt` table	1	5	7	10	7
Total						42

View Functions

`BillListView`

`get(self: BillListView, request: HTTPRequest, household_id: int)`

Get a list of active `Bills` for the authenticated `User` and the given `Household`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, request, household ID	3	3	4	6	12
EOs	Response	1	4	5	7	5
EQs	`Bill` query	1	3	4	6	4
ILFs	None	0	7	10	15	0
EIFs	`Bill` table	1	5	7	10	7
Total						28

`post(self: BillListView, request: HTTPRequest, household_id: int)`

Validate involved `Users` and create a new `Bill` in the given `Household`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, request, household ID	3	3	4	6	12
EOs	Response	1	4	5	7	5
EQs	`User` query	1	3	4	6	4
ILFs	None	0	7	10	15	0
EIFs	`User` table	1	5	7	10	7
Total						28

BillDetailView

`get(self: BillDetailView, request: HTTPRequest, bill_id: int)`

Get the details of an individual `Bill`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, request, bill ID	3	3	4	6	12
EOs	Response	1	4	5	7	5
EQs	`Bill` query	1	3	4	6	4
ILFs	None	0	7	10	15	0
EIFs	`Bill` table	1	5	7	10	7
Total						28

`put(self: BillDetailView, request: HTTPRequest, bill_id: int)`

Update a `Bill`'s details.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, request, bill ID	3	3	4	6	12
EOs	Response	1	4	5	7	5
EQs	`Bill` query	1	3	4	6	4
ILFs	None	0	7	10	15	0
EIFs	`Bill` table	1	5	7	10	7
Total						28

`delete(self: BillDetailView, request: HTTPRequest, bill_id: int)`

Delete a `Bill`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, request, bill ID	3	3	4	6	12
EOs	Response	1	4	5	7	5
EQs	`Bill` query	1	3	4	6	4
ILFs	None	0	7	10	15	0
EIFs	`Bill` table	1	5	7	10	7
Total						28

`ResolvedBillListView`

`get(self: ResolvedBillListView)`

Get a list of resolved `Bills` for the authenticated `User` and the given `Household`.

Domain	Description	Count	Simple	Average	Complex	Product
EIs	Self, request, household ID	3	3	4	6	12
EOs	Response	1	4	5	7	5
EQs	`Bill` query	1	3	4	6	4
ILFs	None	0	7	10	15	0
EIFs	`Bill` table	1	5	7	10	7
Total						28